



HCMC University of Technology, VNU

Software Architecture



Chapter 5. Architecture Styles (cont.)

Lecturer: Thai-Minh Truong, PhD.

Email: thaiminh@hcmut.edu.vn

CHAPTER 5. Architecture Styles



5.6. Service-based Architecture Style

5.7. Micro-service Architecture Style (Self-study)

5.6. Service-based Architecture Style



- Service-based architecture is considered as one of **the most practical architecture styles**, mostly due to its architectural **flexibility**. Although service-based architecture is a distributed architecture, it doesn't have the same level of complexity and cost as other distributed architectures, such as microservices or event-driven architecture, making it a **very popular choice** for many business-related applications.

5.6. Service-based Architecture Style



- The **basic topology** of service-based architecture follows a distributed macro layered structure consisting of a separately deployed user interface, separately deployed remote coarse-grained services, and a monolithic database. This basic topology is illustrated in Figure 13-1

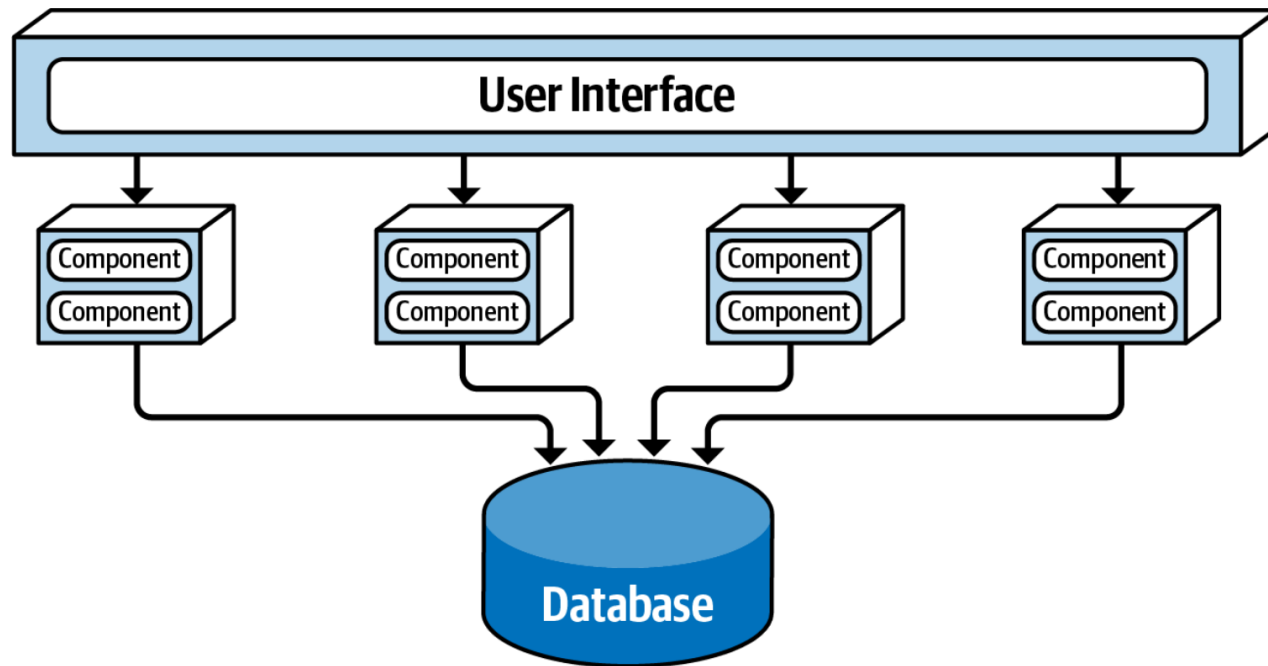


Figure 13-1. Basic topology of the service-based architecture style

5.6. Service-based Architecture Style



- The **number of services** within an application context generally **range between 4 and 12 services**, with the **average** being **about 7 services**.
- In **most cases** there is **only a single instance** of **each domain service** within a service-based architecture.
- However, based on scalability, fault tolerance, and throughput needs, **multiple instances of a domain service can certainly exist**. Multiple instances of a service usually **require** some sort of **load-balancing capability** between the user interface and the domain service.

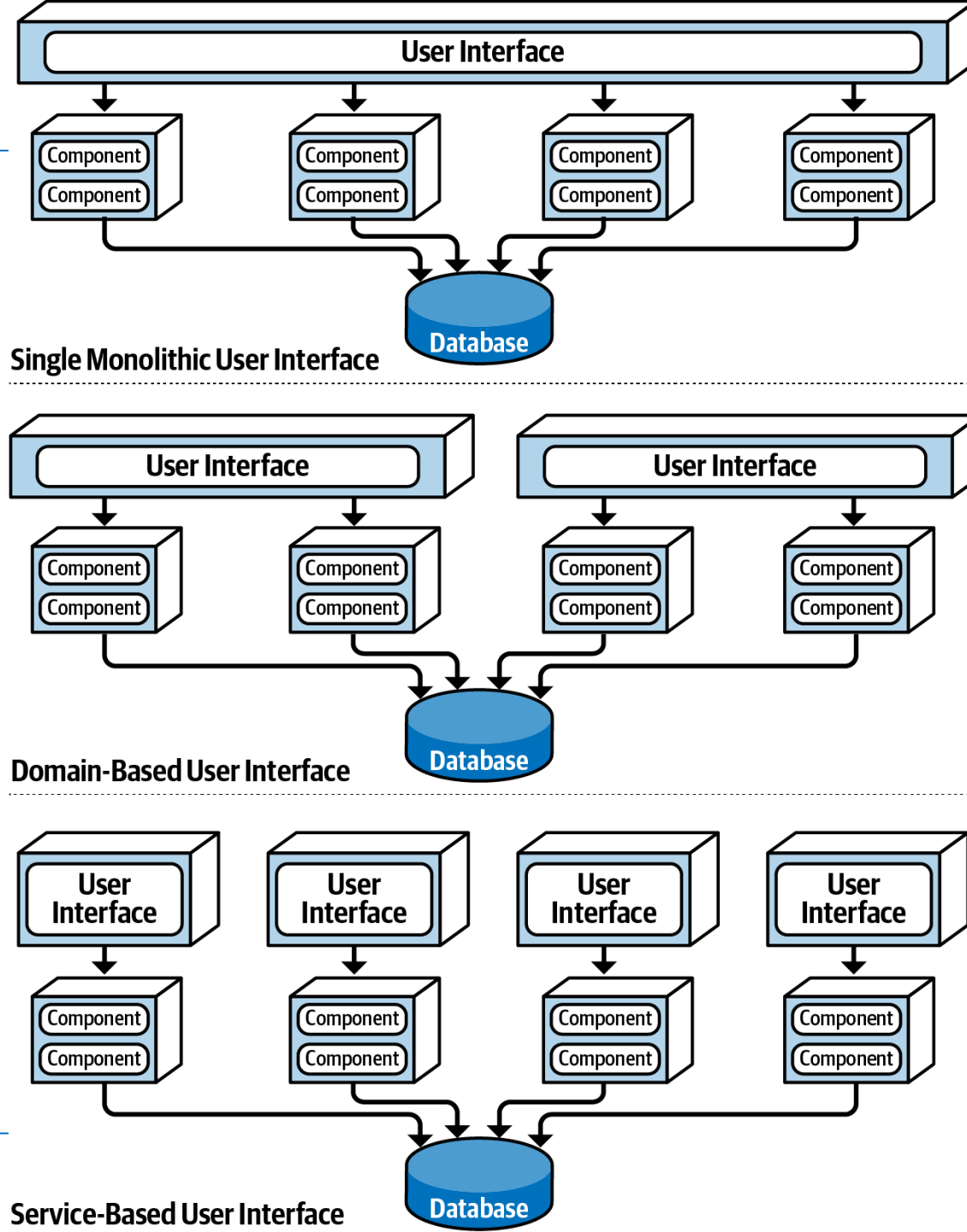
5.6. Service-based Architecture Style



- **Services are accessed remotely** from a user interface using a **remote access protocol**. While REST is typically used to access services from the user interface, messaging, remote procedure call (RPC), or even SOAP could be used as well.
- One important aspect of service-based architecture is that it **typically uses a centrally shared database**.

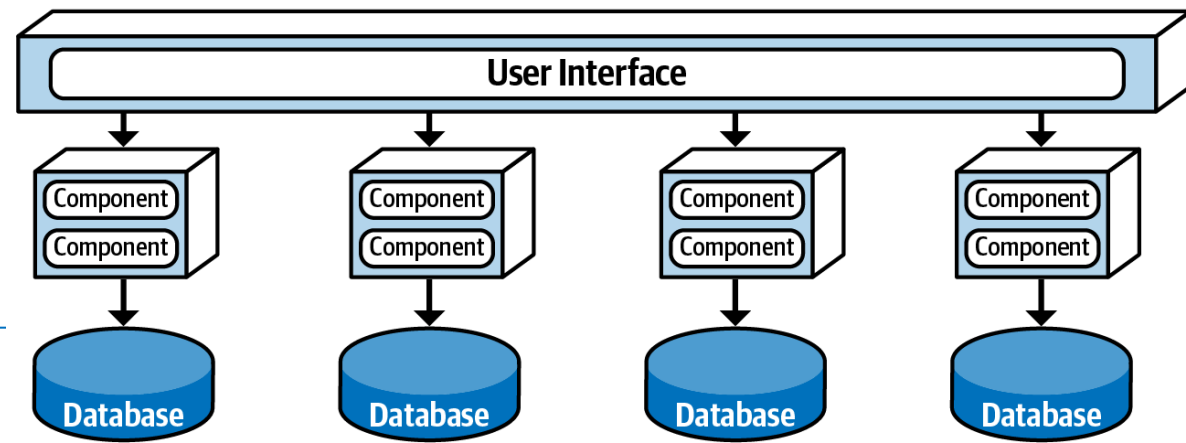
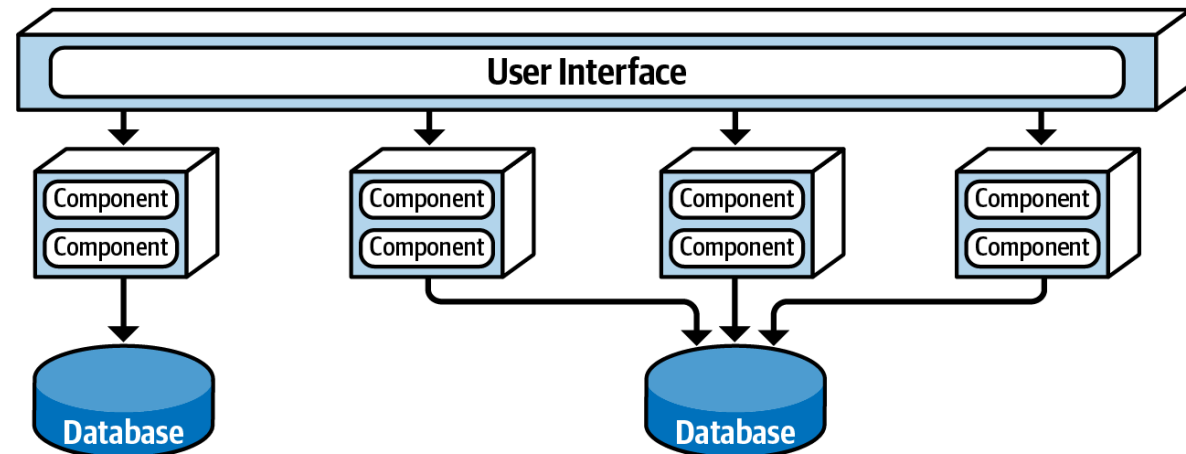
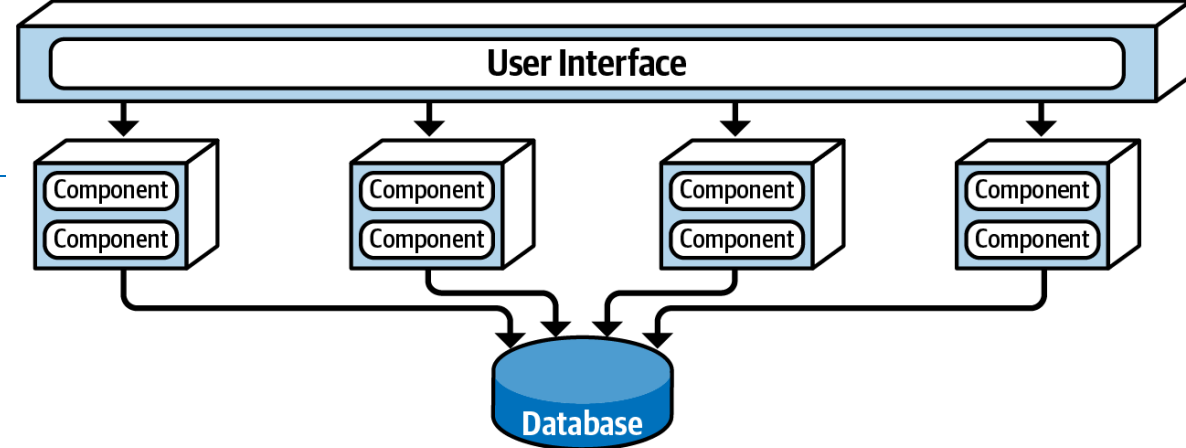
5.6. Service-based Architecture Style

- Topology Variants (user interface variants)



5.6. Service-based Architecture Style

- Topology Variants (database variants)



5.6. Service-based Architecture Style



- **Topology Variants:** It is also possible to add an **API layer** consisting of a **reverse proxy** or **gateway** between the user interface and services, as shown in Figure 13-4.

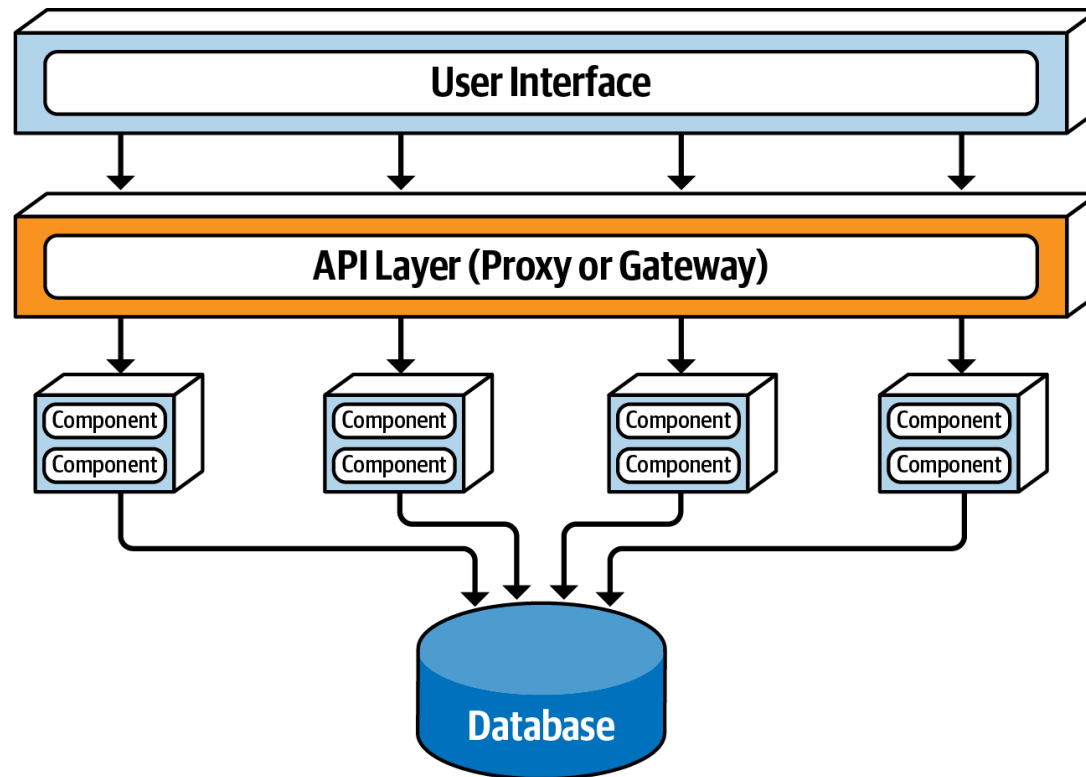
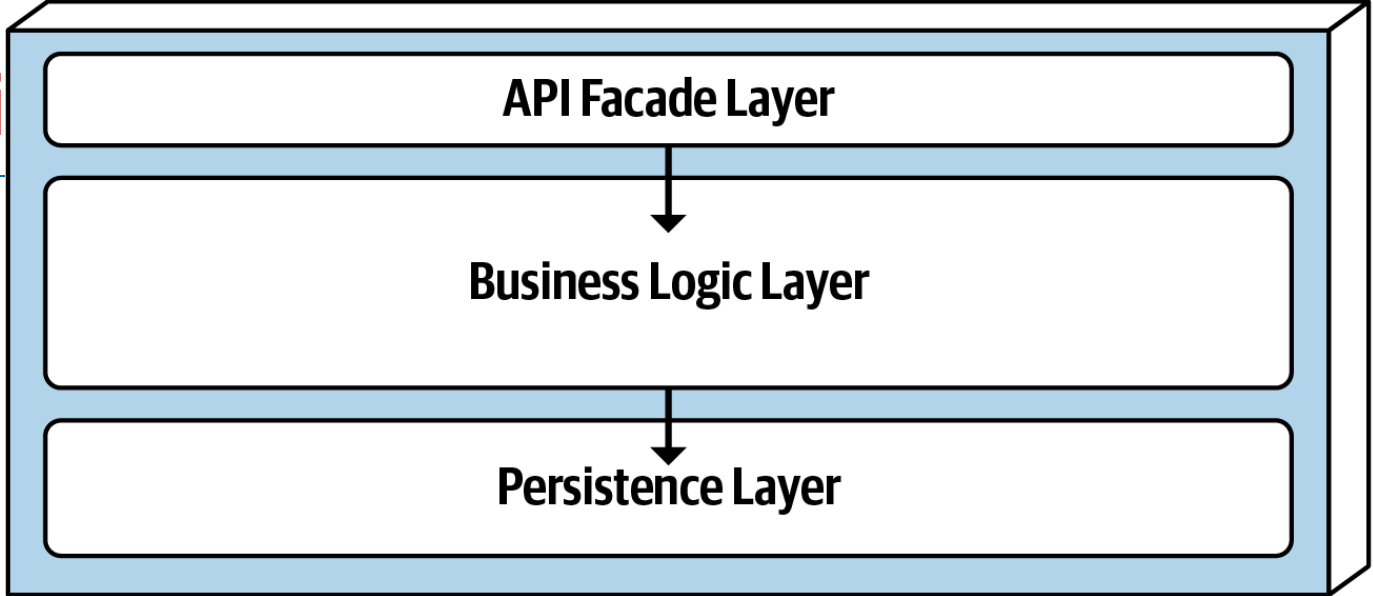


Figure 13-4. Adding an API layer between the user interface and domain services

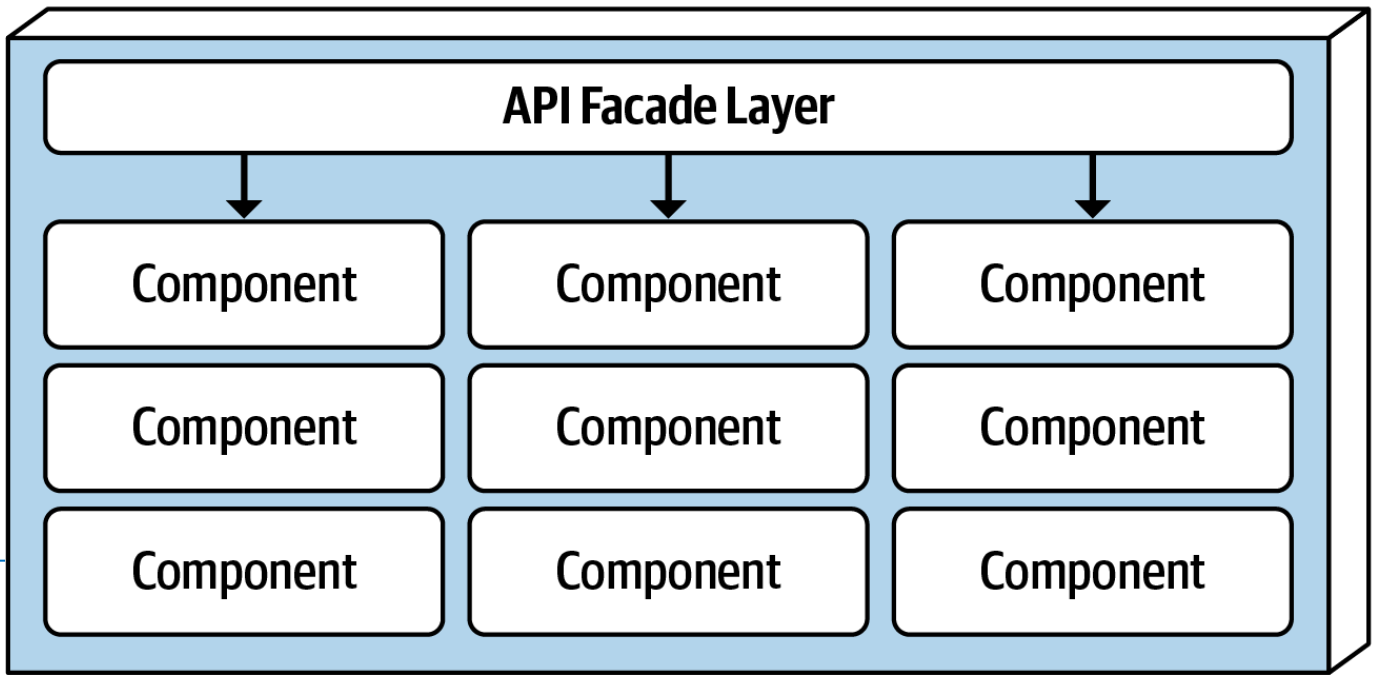
5.6. Servi

Service Design and Granularity



Layered design (technical partitioning)

Domain design (domain partitioning)



5.6. Service-based Architecture Style



- **Service Design and Granularity**
- Domain services:
 - **coarse-grained services**
 - using regular **ACID** (**atomicity, consistency, isolation, durability**) database transactions involving **database commits** and **rollbacks**
 - **ensure database integrity** within a single domain service.
- Highly distributed architectures like microservices:
 - **fine-grained services**
 - using a distributed transaction technique known as **BASE transactions** (**basic availability, soft state, eventual consistency**) that rely on **eventual consistency**
 - **do not support the same level of database integrity as ACID** transactions.

5.6. Service-based Architecture Style



- Database Partitioning

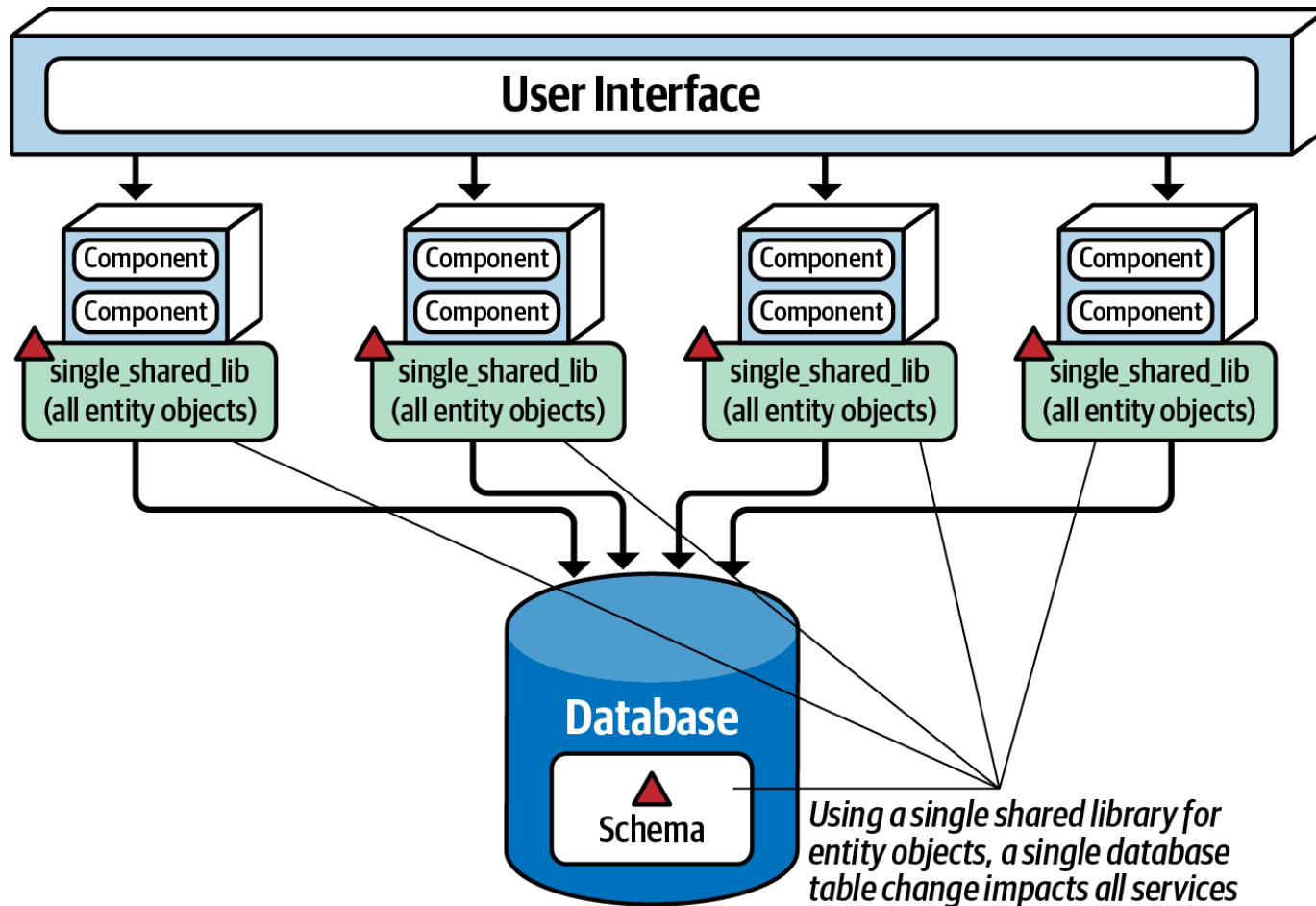


Figure 13-6. Using a single shared library for database entity objects

5.6. Service-based Architecture Style



- Database Partitioning

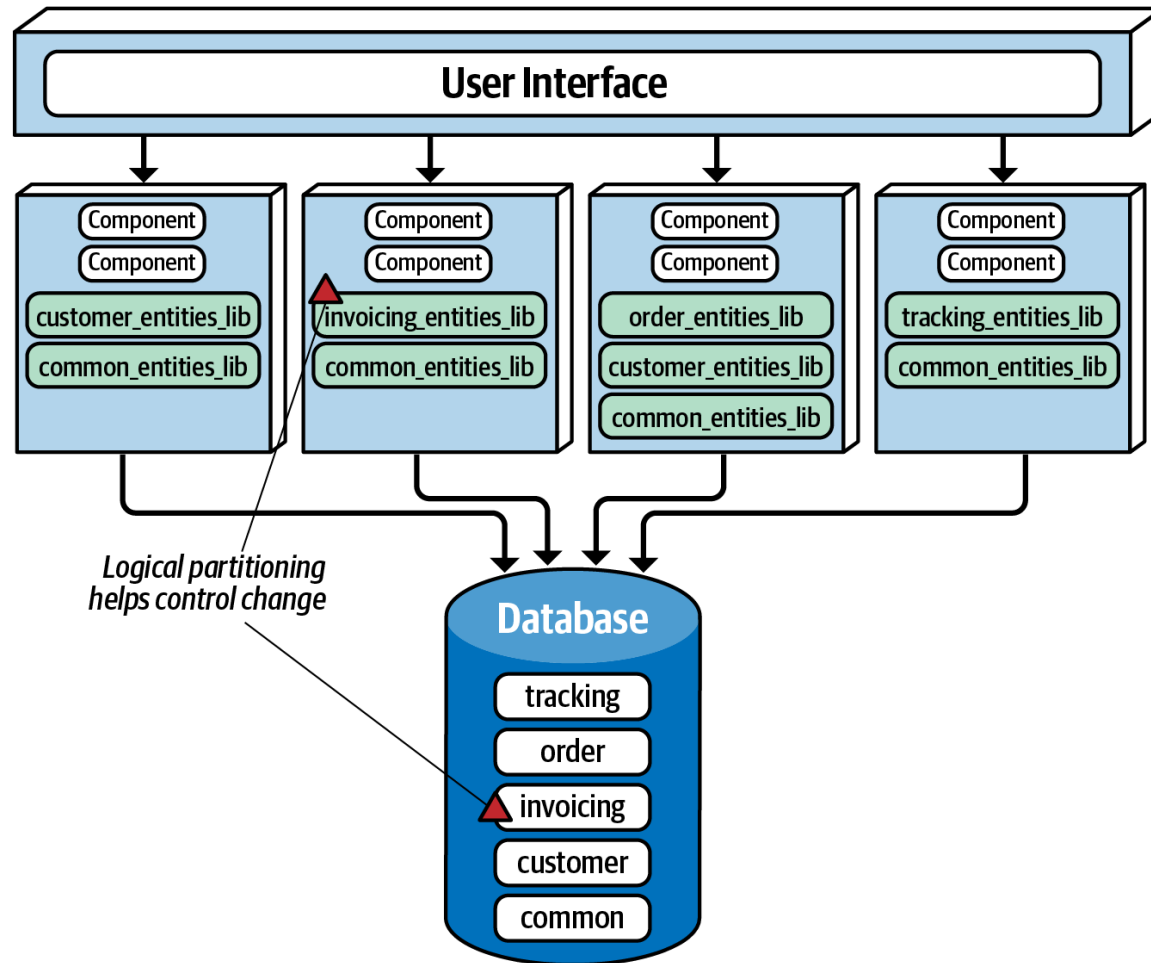


Figure 13-7. Using multiple shared libraries for database entity objects

5.6. Service-based Architecture Style



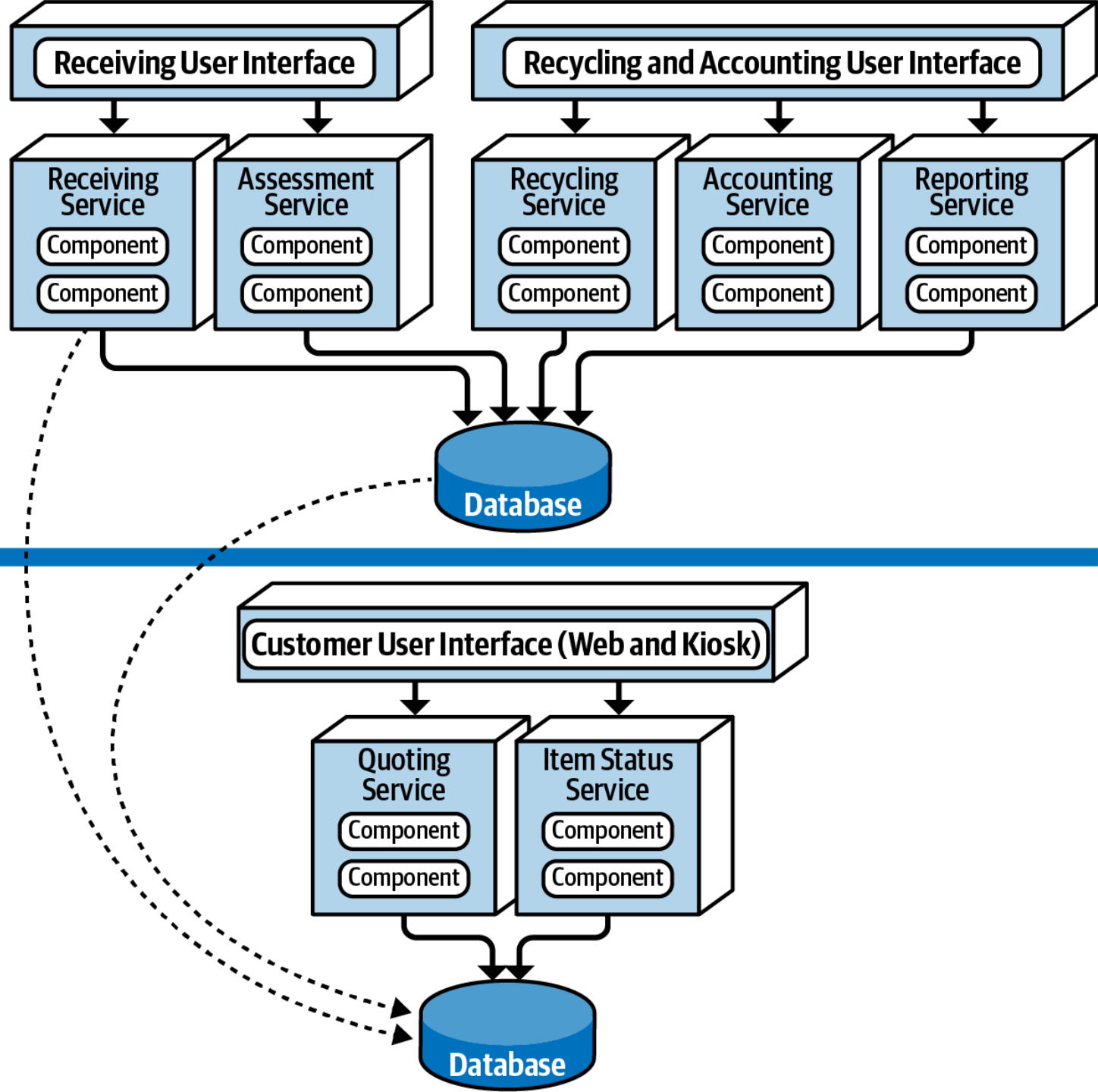
- **Example architecture: an electronics recycling system**
 - To **recycle old electronic devices** (such as an iPhone or Galaxy cell phone).
 - First, the customer asks the company (via a website or kiosk) **how much money** they can get for the old electronic device (called **quoting**).
 - If satisfied, the **customer** will **send** the electronic device to the **recycling company**, which in turn will **receive** the physical device (called **receiving**).
 - Once received, the recycling company will then **assess** the device to **determine** if the device is in good working **condition** or not (called **assessment**).

5.6. Service-based Architecture Style



- **Example architecture:** an electronics recycling system
 - If the device is in good working condition, the company will **send** the customer the **money** promised for the device (called **accounting**).
 - Through this process, the customer can go to the website at any time to **check** on the **status** of the item (called **item status**).
 - Based on the assessment, the **device** is then **recycled** by either safely destroying it or reselling it (called **recycling**).
 - Finally, the company periodically runs ad hoc and **scheduled financial** and **operational reports** based on recycling activity (called **reporting**).

5.6. Serv



- Example architecture

5.6. Service-based Architecture Style



- **Example architecture:** an electronics recycling system
 - Each domain area is implemented as a separate service
 - User interfaces are federated into their respective domains
 - Two separate databases are external and internal operations
 - This example illustrates many of the benefits of the service-based architecture approach: scalability, fault tolerance, and security (data and functionality protection and access), in addition to agility, testability, and deployability.

5.6. Service-based Architecture Style



- Architecture Characteristics Ratings

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★
Evolutionary	★★★
Fault tolerance	★★★★
Modularity	★★★★
Overall cost	★★★★
Performance	★★★
Reliability	★★★★
Scalability	★★★
Simplicity	★★★
Testability	★★★★

5.6. Service-based Architecture Style



- **Architecture Characteristics Ratings**
- **Domain-partitioned**, meaning the structure is driven by the domain.
- Can have **multiple quanta** (separate, deployable units) due to federated components
- **High score** in **agility** (faster change), **testability**, **deployability**, **fault tolerance**, and **simplicity/cost**.
- **Lower score** in **scalability** and **elasticity** due to its coarse-grained nature.
- Generally **more reliable** than other distributed architectures.

5.6. Service-based Architecture Style



- **When to Use This Architecture Style**
- A **pragmatic choice** due to its **flexibility** and **good characteristic ratings**.
- A **good fit for domain-driven** design due to its coarse-grained, domain-scoped services.
- Helps **maintain ACID** transactions **better** than other distributed architectures.
- Offers a **good balance** between **modularity** and **avoiding the complexities** of fine-grained services.
- A **popular choice** for many **business-related applications** due to its balance of flexibility, simplicity, and functionality.
- Offers a **good alternative to more complex distributed architectures** while still achieving many of the desired benefits

5.7. Micro-service Architecture Style

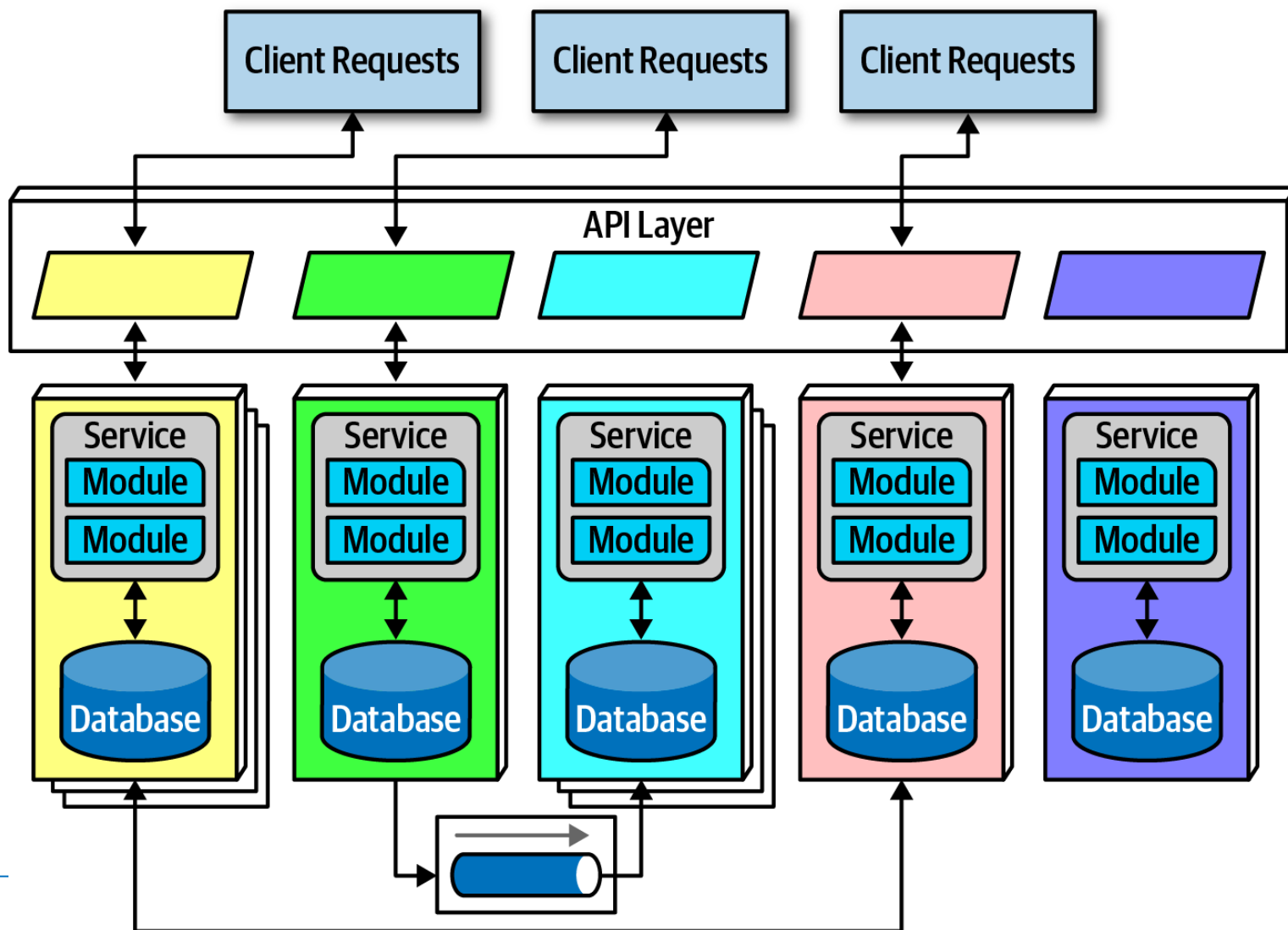


- Microservices is an **extremely popular** architecture style that has gained significant momentum in recent years.
- Microservices is heavily inspired by the ideas in **domain-driven design (DDD)**, a logical design process for software projects.
- One concept in particular from DDD, **bounded context**, represents a **decoupling style**.
- While **reuse** is beneficial, the negative trade-off of reuse is **coupling**.
- If the architect's goal requires high degrees of decoupling, then they favor duplication over reuse. The **primary goal of microservices** is **high decoupling**, physically modeling the logical notion of bounded context.

5.7. Micro-service Architecture Style



- Topology



5.7. Micro-service Architecture Style



- **Distributed:**
- Microservices form a **distributed architecture**: **each service runs in its own process**, which originally implied a physical computer but quickly evolved to virtual machines and containers.
- **Separating** each **service** into its own process **solves** all the **problems** brought on **by sharing**.
- **Performance** is often the **negative side effect** of the distributed nature of microservices. Network calls take much longer than method calls, and security verification at every endpoint adds additional processing time.
- Requiring architects to **think carefully** about the **implications of granularity** when designing the system.

5.7. Micro-service Architecture Style



- **Bounded Context:** each service models a domain or workflow.
 - **Each service includes everything necessary** to operate within the application, including classes, other subcomponents, and database schemas.
 - **Each service** is meant to represent a **domain** or **subdomain**.
- **Granularity:**
 - Architects **struggle to find the correct granularity for services**, and often make the mistake of making their services too small, which requires them to build communication links back between the services to do useful work.
 - The **purpose of service boundaries** in microservices is to capture a **domain** or **workflow**.

5.7. Micro-service Architecture Style



- **Granularity:**
- Some **guidelines** to help **find** the **appropriate boundaries**:
 - ***Purpose***: Ideally, **each microservice** should be **extremely functionally cohesive**, contributing **one significant behavior** on behalf of the overall application.
 - ***Transactions***: **Bounded contexts** are **business workflows**, and often the entities that need to cooperate in a transaction show architects a good service boundary.
 - ***Choreography***: **Iteration** is the **only way to ensure good service design**. Architects rarely discover the perfect granularity, data dependencies, and communication styles on their first pass. However, after iterating over the options, an architect has a good chance of refining their design.

5.7. Micro-service Architecture Style



- **Data Isolation:**
- Another requirement of microservices, driven by the bounded context concept, is **data isolation**. Many other architecture styles use a single database for persistence. However, microservices tries to **avoid all kinds of coupling**, including shared schemas and databases used as integration points.
- **API Layer:**
- Most pictures of microservices include an **API layer** sitting between the consumers of the system (either user interfaces or calls from other systems), but it is **optional**. It is **common** because it **offers a good location** within the architecture to **perform useful tasks**, either via indirection as a proxy or a tie into operational facilities.

5.7. Micro-service Architecture Style



- **Operational Reuse:**
- In the **sidecar pattern**, the **common operational concerns appear** within each service as a **separate component**, which can be **owned** by either **individual teams** or a **shared infrastructure team**. The sidecar component handles all the operational concerns that teams benefit from coupling together. Thus, when it comes time to upgrade the monitoring tool, the shared infrastructure team can update the sidecar, and each microservices receives that new functionality.

5.7. Micro-service Architecture Style

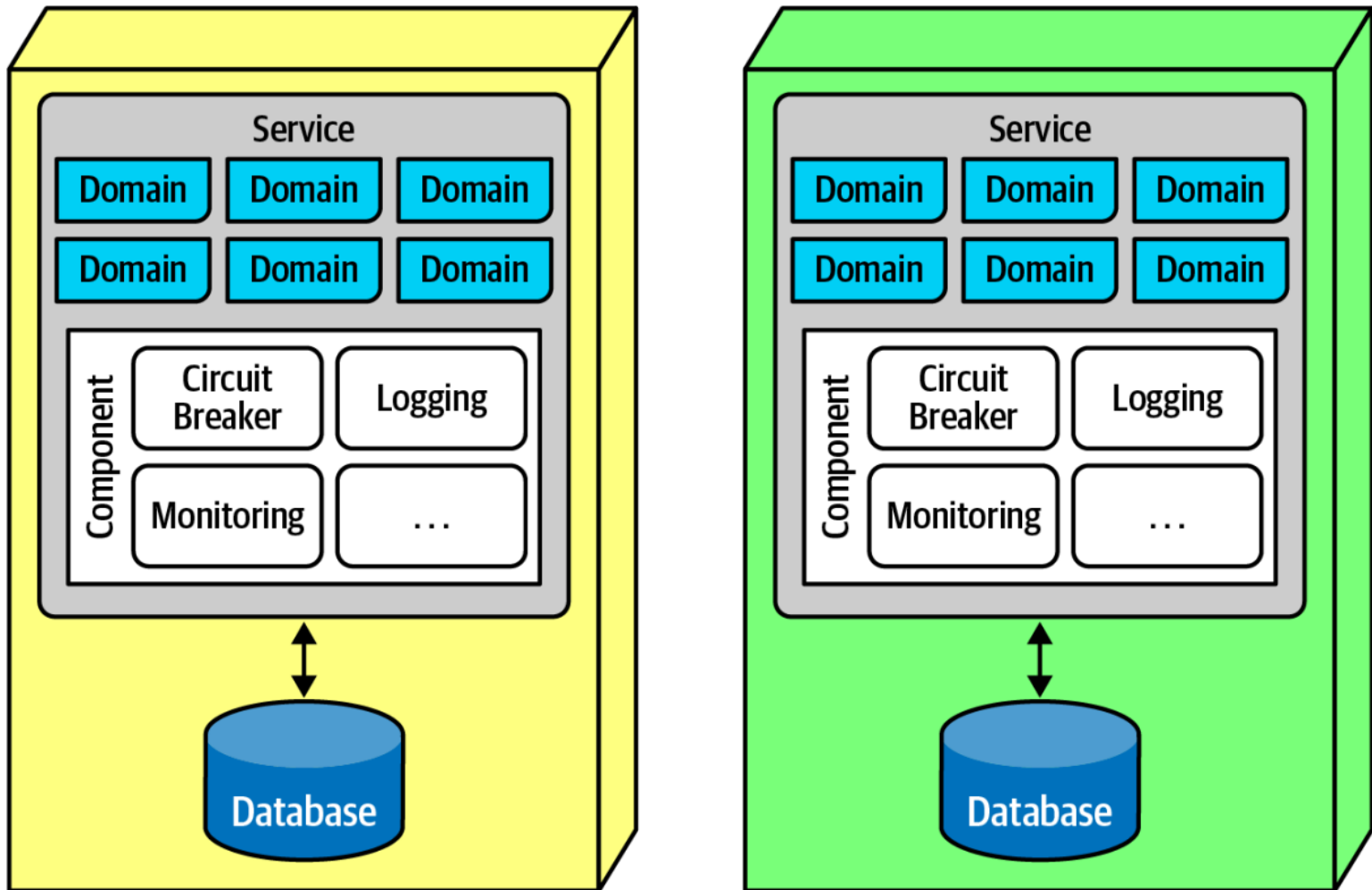


Figure 17-2. The sidecar pattern in microservices

5.7. Micro-service Architecture Style



- **Operational Reuse:**
- A **service mesh**, allowing **unified control across the architecture for concerns** like logging and monitoring. The common sidecar components connect to form a consistent operational interface across all microservices.
- Each **sidecar wires** into the **service plane**, which forms the consistent interface to each service.

5.7. Micro-service Architecture Style

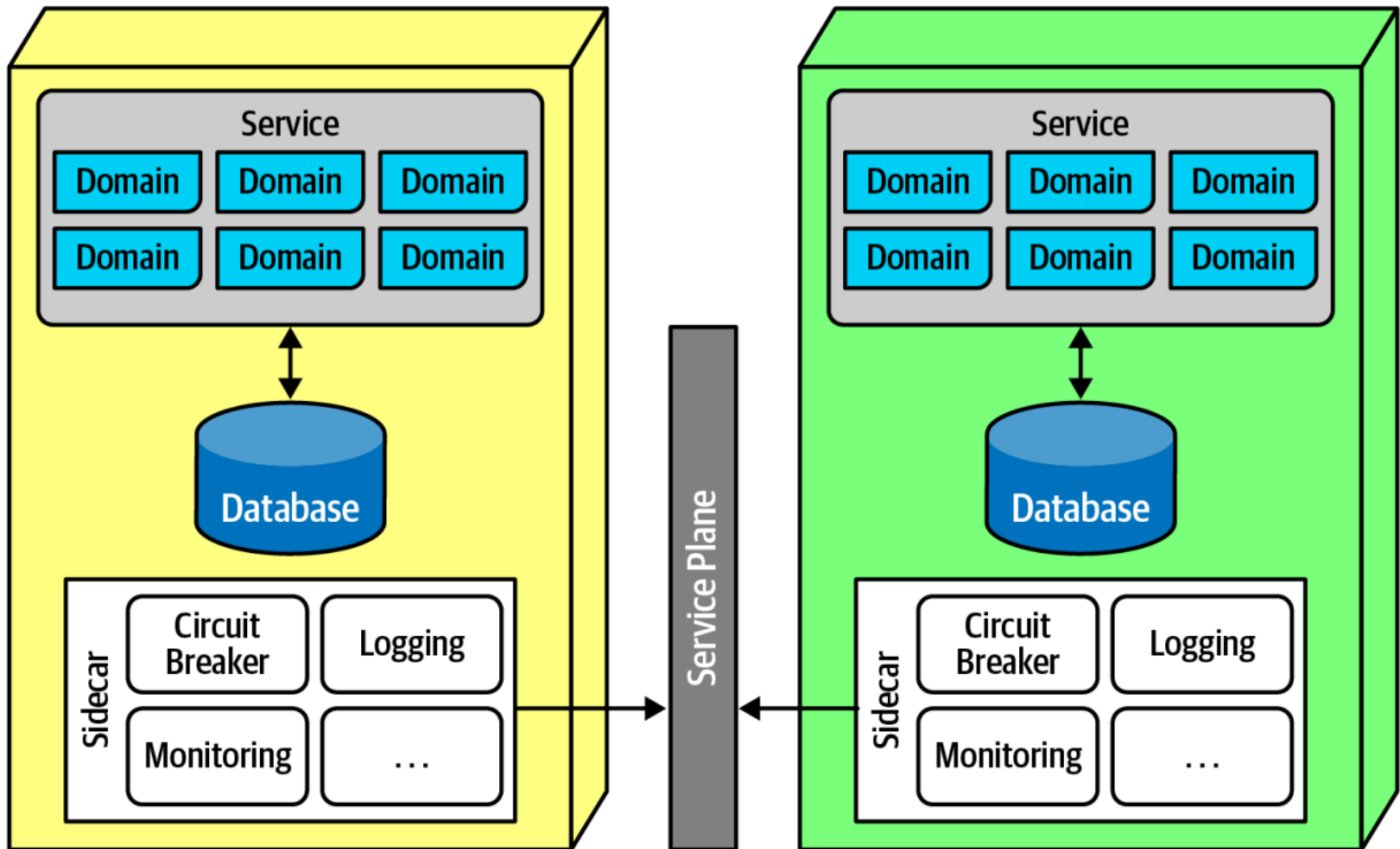


Figure 17-3. The service plane connects the sidecars in a service mesh

5.7. Micro-service Architecture Style



- **Operational Reuse:**
- The **service mesh** itself **forms** a **console** that **allows** developers holistic **access to services**
- **Each service forms a node** in the overall **mesh**. The **service mesh** forms a console that allows teams to **globally control operational coupling**, such as monitoring levels, logging, and other cross-cutting operational concerns.
- **Service discovery** is used as a way to build **elasticity** into microservices architectures. Rather than invoke a single service, a request goes through a **service discovery** tool, which can **monitor** the **number** and **frequency of requests**, as well as **spin up new instances of services** to **handle scale** or **elasticity concerns**. **Service discovery** is often **included** in the **service mesh**, making it part of every microservice. The **API layer** is often used to **host service discovery**, allowing a single place for user interfaces or other calling systems to find and create services in an elastic, consistent way.

5.7. Micro-service Architecture Style



- Operational Reuse:

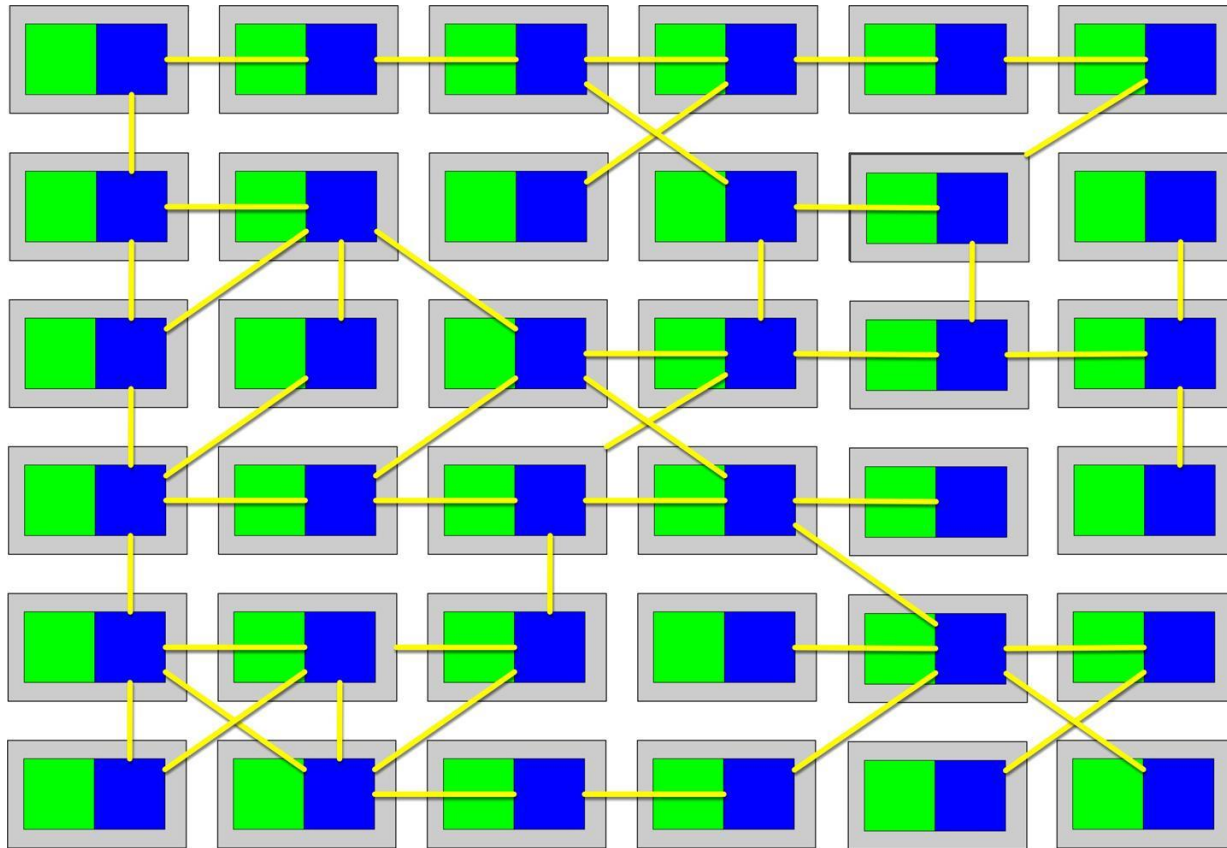


Figure 17-4. The service mesh forms a holistic view of the operational aspect of microservices

5.7. Micro-service Architecture Style



- **Frontends:**
- In fact, the original vision for microservices included the user interface as part of the bounded context, faithful to the principle in DDD. However, practicalities of the partitioning required by web applications and other external constraints make that goal difficult.
- Thus, two styles of user interfaces commonly appear for microservices architectures:
 - the first uses a **monolithic user interface**,
 - the second uses the **microfrontend pattern**.

5.7. Micro-service Architecture Style

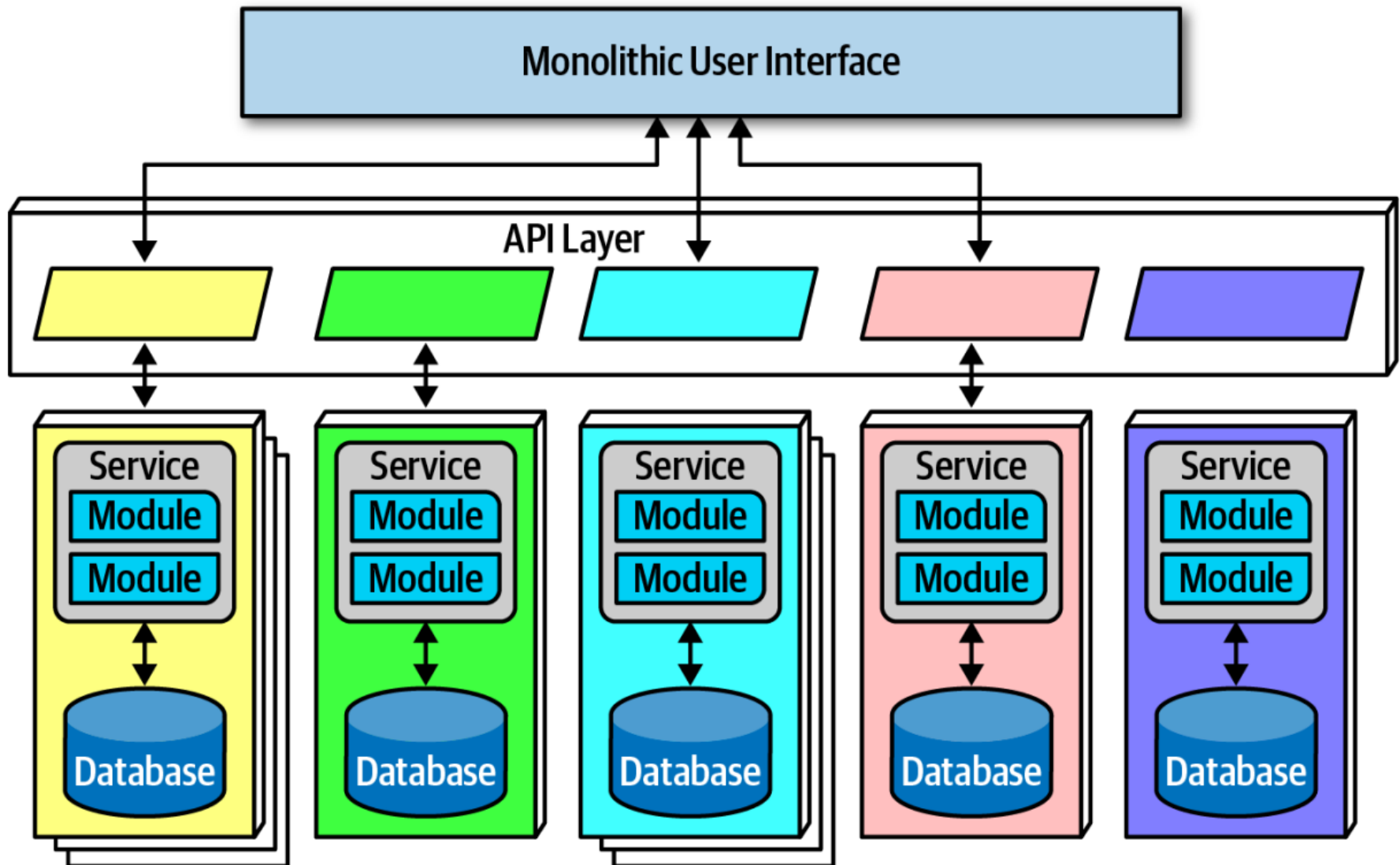


Figure 17-5. Microservices architecture with a monolithic user interface

5.7. Micro-service Architecture Style

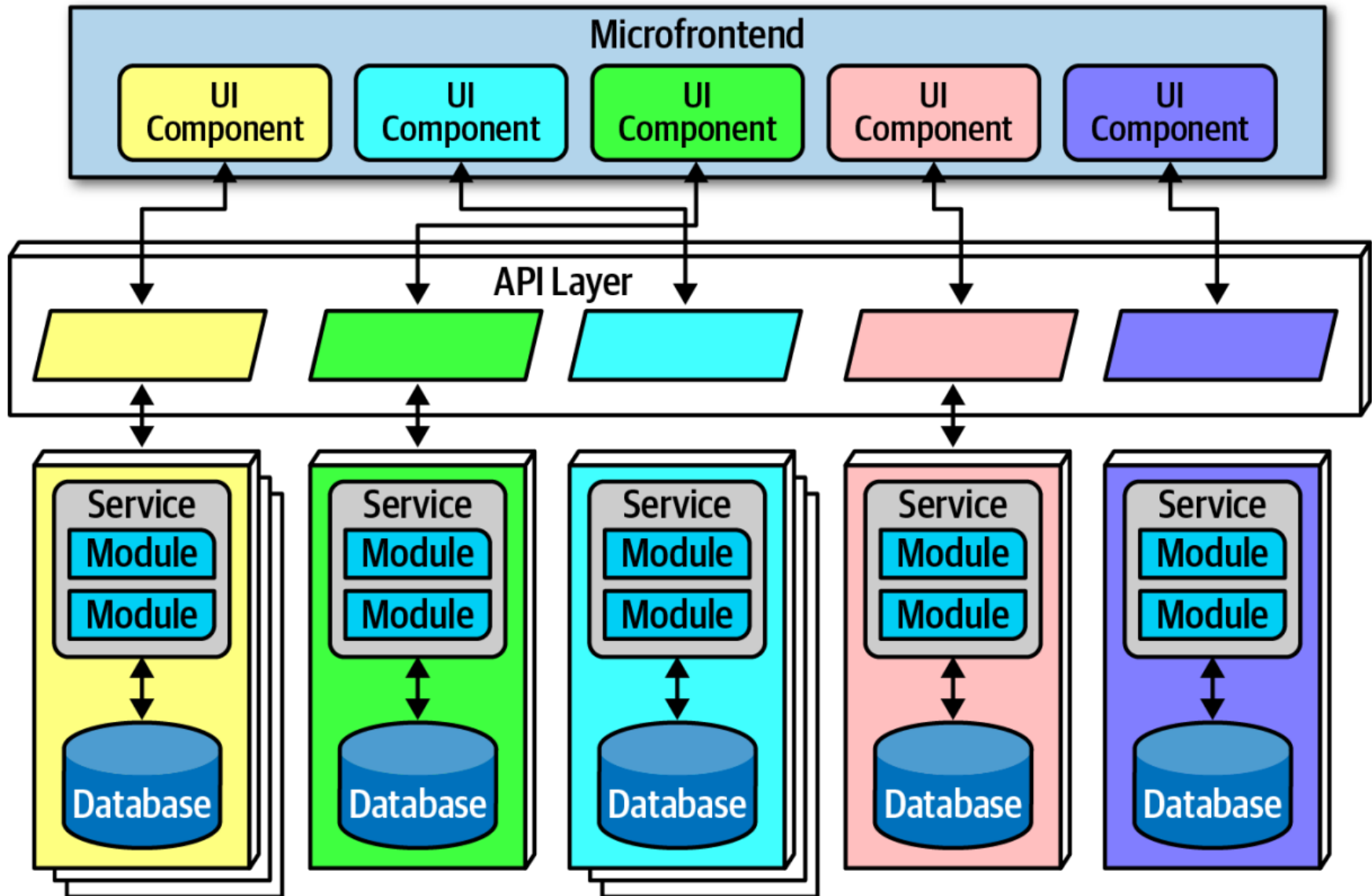


Figure 17-6. Microfrontend pattern in microservices

5.7. Micro-service Architecture Style



- **Communication:**
- Microservices architectures typically utilize **protocol-aware heterogeneous interoperability**. This term will be broken down as follows:
 - **Protocol-aware:** Services **must know** (or discover) which **protocol** to use to call other services.
 - **Heterogeneous:** Because each service may be written in a different technology stack. Heterogeneous suggests that microservices **fully supports polyglot environments**, where **different services use different platforms**.
 - **Interoperability:** Describes **services calling one another**. While architects in microservices try to discourage transactional method calls, services commonly call other services via the network to collaborate and send/receive information.
- **Enforced Heterogeneity**

5.7. Micro-service Architecture Style



- **Choreography and Orchestration:**
- **Choreography** utilizes the same communication style as a **broker event-driven architecture**. In other words, **no central coordinator** exists in this architecture, respecting the bounded context philosophy. Each service calls other services as needed, without a central mediator.

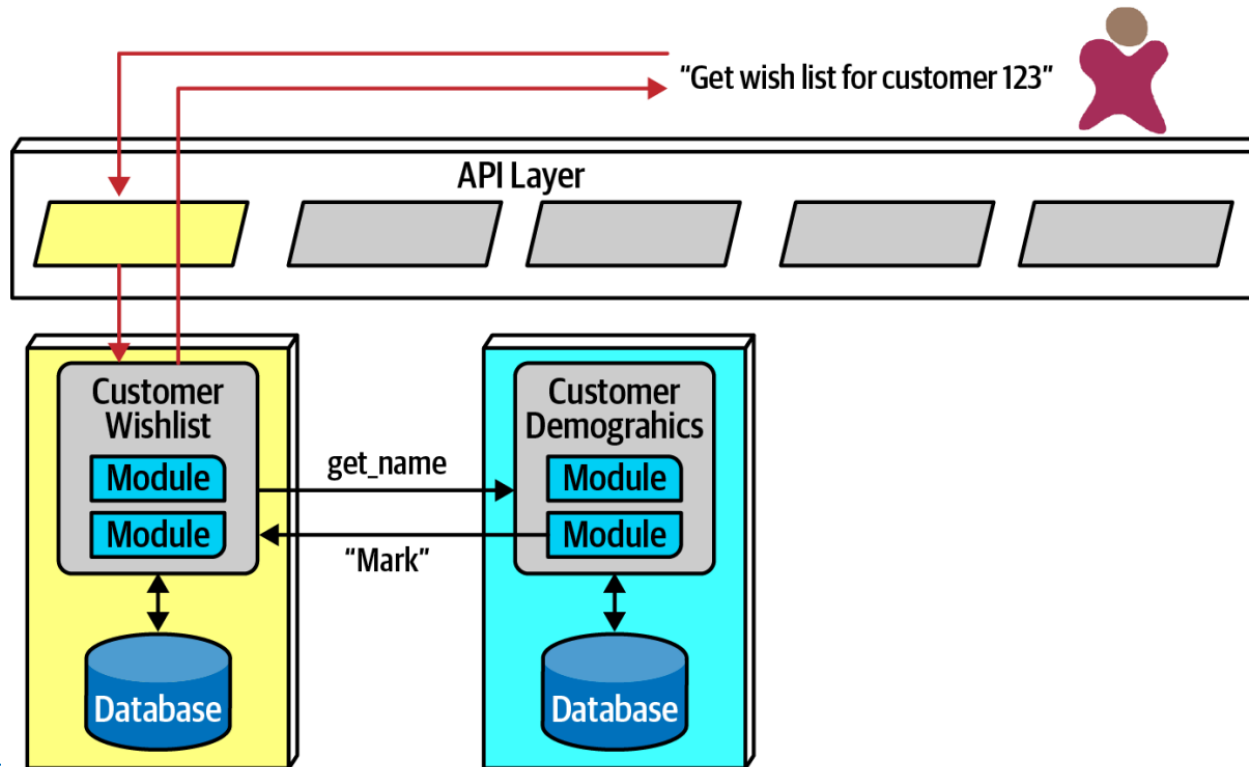


Figure 17-7. Using choreography in microservices to manage coordination

5.7. Micro-service Architecture Style



- **Choreography and Orchestration:**
- Because microservices architectures **don't include a global mediator** like other service-oriented architectures, if an architect needs to coordinate across several services, they can create their own **localized mediator**.

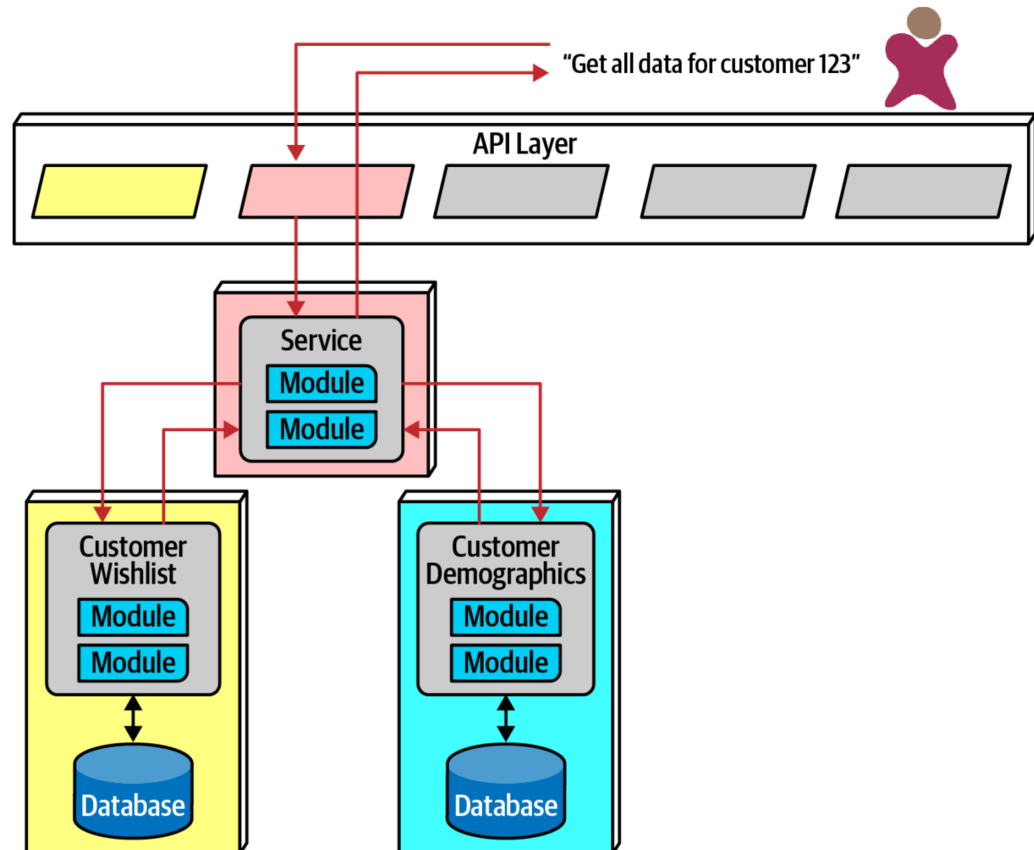


Figure 17-8. Using orchestration in microservices

5.7. Micro-service Architecture Style



- **Choreography and Orchestration:**
- In **choreography**, the architect preserves the **highly decoupled** philosophy of the architecture style, thus reaping maximum benefits touted by the style. However, **common problems** like **error handling** and **coordination** become more **complex** in choreographed environments.

5.7. Micro-service Architecture Style

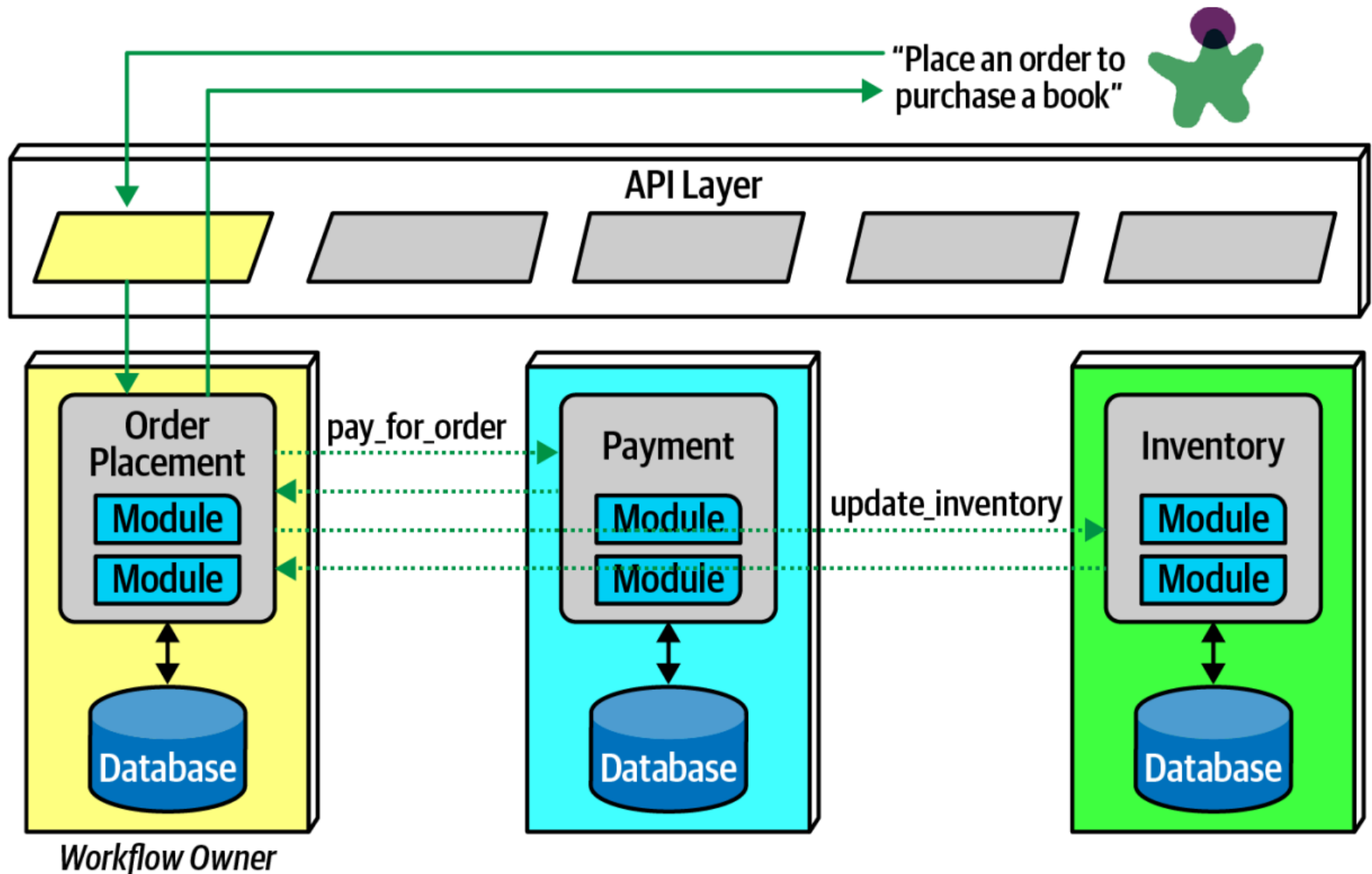


Figure 17-9. Using choreography for a complex business process

5.7. Micro-service Architecture Style

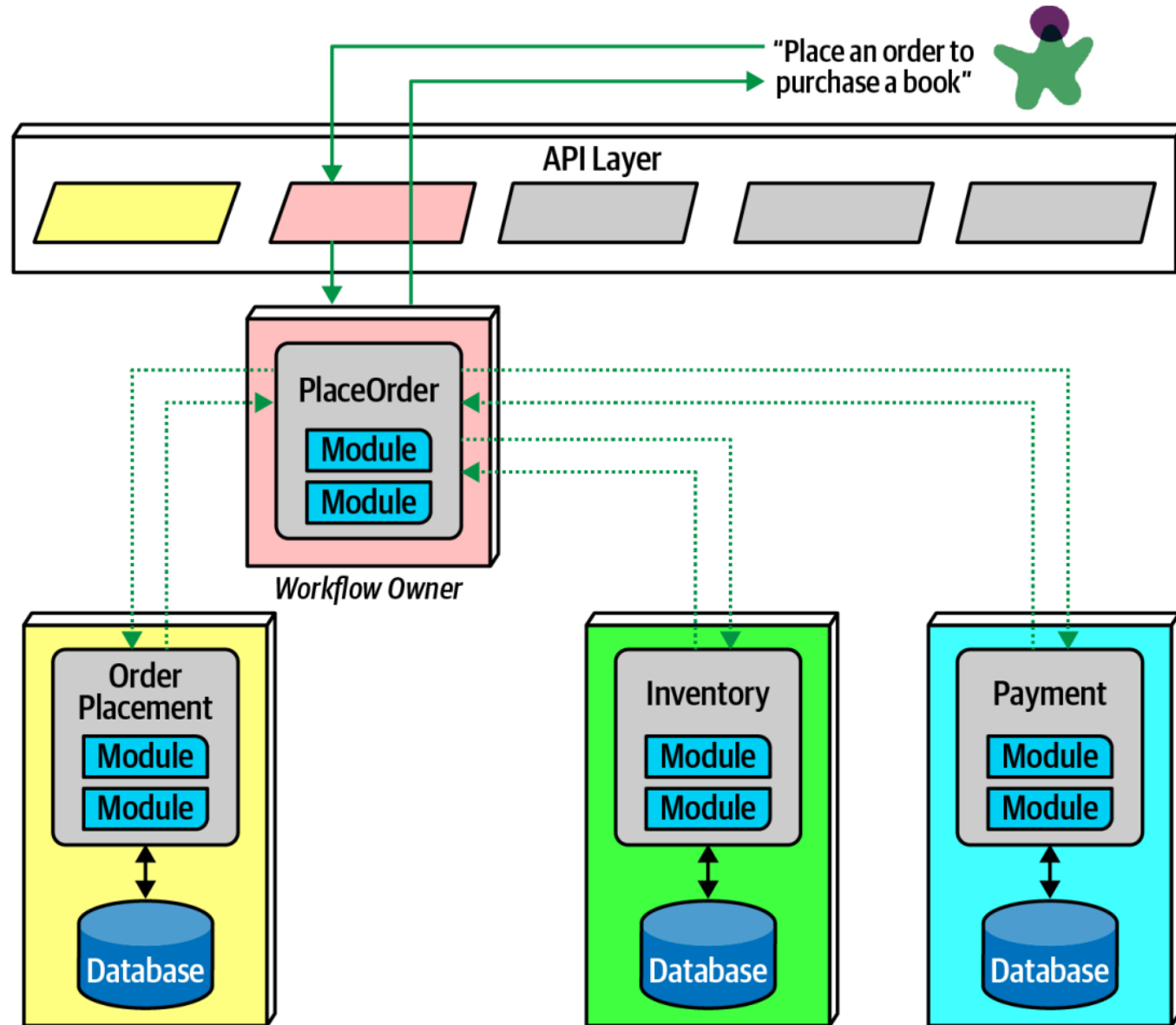


Figure 17-10. Using orchestration for a complex business process

5.7. Micro-service Architecture Style



- **Transactions and Sagas:**
- In microservices, **extreme decoupling** often **encounters** the **problem** of how to do **transactional coordination** across services.
- **Atomicity** becomes a **problem** in **distributed architectures**, because the decoupling in the architecture encourages the same level for the databases.
- Building **transactions across service boundaries violates** the **core decoupling principle** of the microservices architecture.
- *Don't do transactions in microservices—fix granularity instead!*
- **Exceptions** always exist. In those situations, **patterns** exist to **handle transaction orchestration**, with serious trade-offs.
- A **popular distributed transactional pattern** in microservices is the **saga pattern**.

5.7. Micro-service Architecture Style

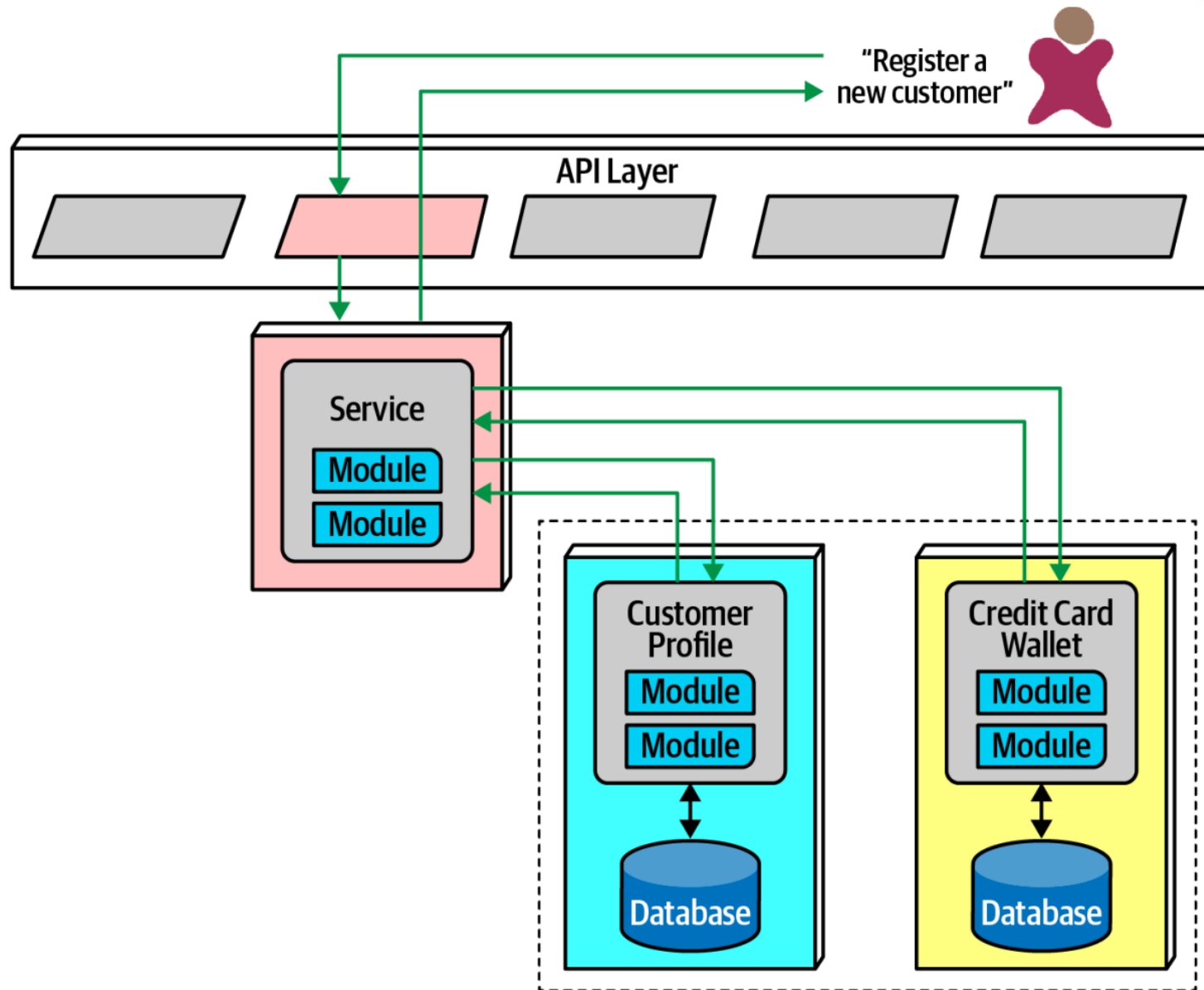


Figure 17-11. The saga pattern in microservices architecture

5.7. Micro-service Architecture Style

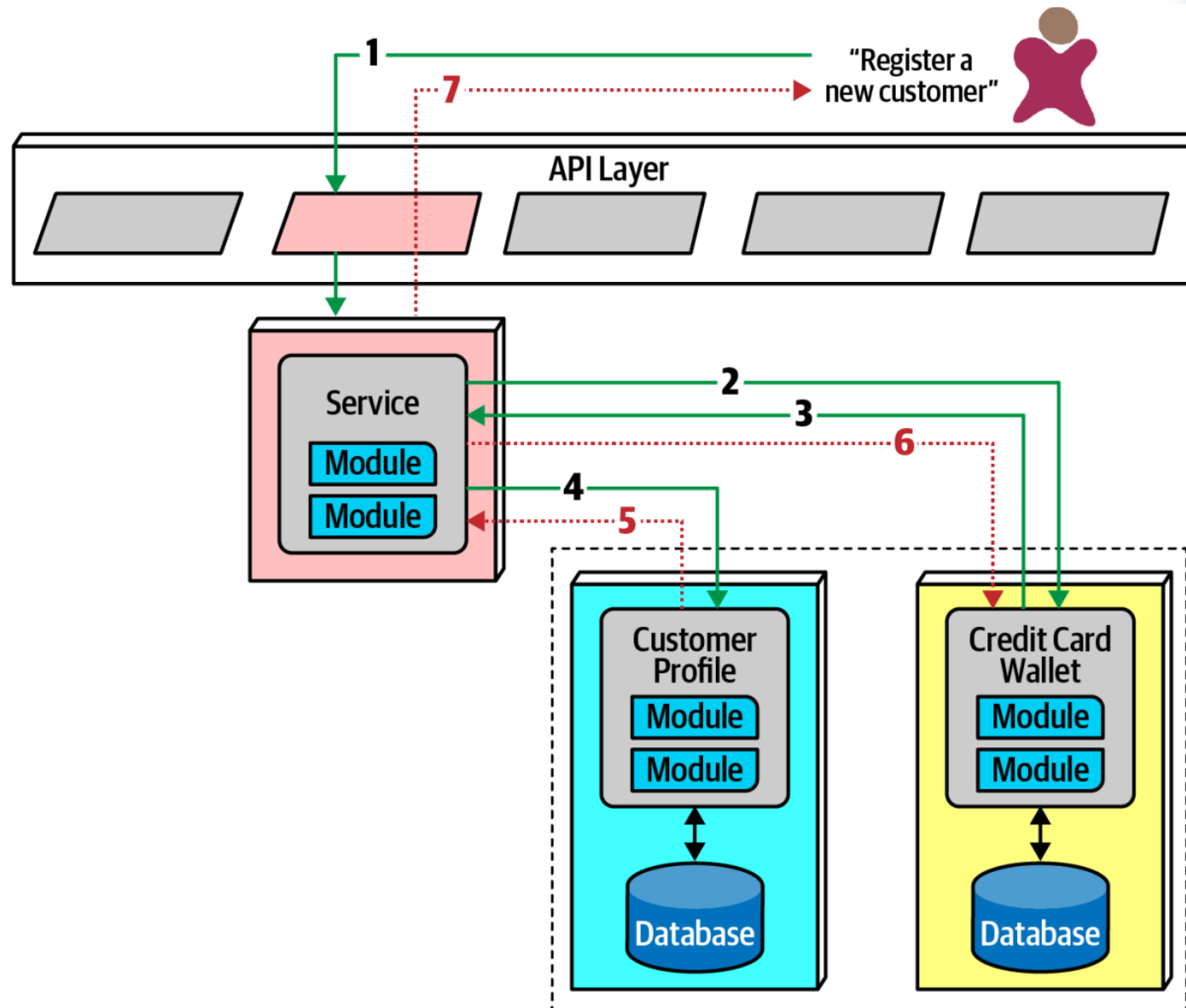


Figure 17-12. Saga pattern compensating transactions for error conditions

5.7. Micro-service Architecture Style



- **Transactions and Sagas:**
- In Figure 17-12, if the **first part** of the transaction **succeeds**, yet the **second part fails**, the **mediator must send a request** to **all** the **parts** of the transaction that were **successful** and tell them to **undo** the **previous request**. This style of transactional coordination is called a **compensating transaction framework**. Developers implement this pattern by usually having each request from the mediator enter a pending state until the mediator indicates overall success.
- While it is possible for architects to build transactional behavior across services, it goes against the reason for choosing the microservices pattern. Exceptions always exist, so the best advice for architects is to **use the saga pattern sparingly**.
- *A few transactions across services is sometimes necessary; if it's the dominant feature of the architecture, mistakes were made!*

5.7. Micro-service Architecture Style



- Architecture Characteristics Ratings:

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★★★★
Evolutionary	★★★★★
Fault tolerance	★★★★
Modularity	★★★★★
Overall cost	★
Performance	★★
Reliability	★★★★
Scalability	★★★★★
Simplicity	★
Testability	★★★★



Q&A